

CEPHALOPODAI | Intelligent Design | LLM Interface

Local Core System Setup and Installation Guide

Project Introduction & Systems Philosophy

Welcome to the deployment framework for **CephalopodAI**, a self-contained, stateless, edge-optimized terminal designed to act as a highly collaborative local workbench. This system bridges the gap between local large language models (LLMs), live external data retrieval, and active workspace rendering. By combining the local execution power of Ollama with the optimized weights of the Gemma 4:e2b model and our custom software layer, this framework allows you to run a heavyweight, intelligent local pipeline safely and completely isolated on your home computer—flawlessly operating with or without an active internet connection.

This release marks the foundational development step for the CephalopodAI platform. It is engineered upon a core principle of absolute intellectual sovereignty: your machine, your data, your rules. Unlike cloud-based AI services that harvest telemetry and user input, this architecture is a 100% locally-contained environment. This ensures that sensitive intellectual property, proprietary data, and creative works remain permanently shielded from external monitoring.

Our mission is to freely distribute this operational capability to the world. The architecture is intentionally decoupled from corporate API paywalls and high-end enterprise hardware restrictions, rendering it universally accessible. Whether you are a legal professional or corporate researcher requiring absolute, air-gapped confidentiality for proprietary data, or an independent builder in the far reaches of the world cobbling together an AI workstation from spare parts and older gaming hardware, CephalopodAI delivers professional-grade computational intelligence directly to your desktop.

While many evolutionary updates are planned for this ecosystem, the following document outlines the exact, step-by-step instructions required to establish your initial local repository, configure your backend utility server, and initialize your frontend control room.

SYSTEM_SETUP_AND_INSTALLATION_GUIDE.md

Project Framework: CEPHALOPODAI (Intellectual Freedom Terminal) Target Environment: Microsoft Windows 10 / 11 (64-bit Architecture) Optimized Engine: Gemma 4:e2b running locally

This guide is a comprehensive, step-by-step blueprint designed for users installing this system for the first time. Follow these exact operational steps to configure your local machine into a high-performance, 100% offline AI workstation.

SECTION 1: SYSTEM HARDWARE VERIFICATION

Before downloading any software, confirm your PC meets or exceeds the baseline specifications verified for this deployment:

- **Processor (CPU):** Intel Core i5-10600K @ 4.10 GHz (or equivalent multi-core processor).
- **System Memory (RAM):** 32 GB DDR4 RAM.
- **Graphics Card (GPU):** NVIDIA GeForce GTX 1080 Ti with 11 GB of dedicated VRAM.
- **Storage Space:** A Solid State Drive (SSD) with at least 20 GB to 30 GB of unallocated free space.

Hardware Note: The 11 GB of VRAM on the graphics card is critical. It allows the system to load the entire weights matrix of the Gemma model directly into high-speed GPU memory, ensuring instant response times and preventing system lockups during massive token sessions.

SECTION 2: DOWNLOADING AND SETTING UP OLLAMA (The AI Engine)

Ollama is the engine that allows you to run raw AI models completely offline on your own hardware without sending data to third-party servers.

1. **Go Online:** Open your web browser and navigate to the official website: <https://ollama.com/>.
2. **Download:** Click the large **Download for Windows** button. This downloads an installer file named `OllamaSetup.exe`.
3. **Install:** Go to your Downloads folder and double-click `OllamaSetup.exe`. Follow the on-screen prompt windows. Click **Install** and let the process finish.
4. **Verify Running State:** Once installed, Ollama runs as a background service. Look at your Windows taskbar tray (bottom right corner of your screen near the clock) and look for the llama icon to confirm it is active.
5. **Download the Gemma Model:**

- a. Click your Windows Start button, type `cmd`, and press Enter to open the **Command Prompt**.
- b. To pull down the specific model optimized for this interface, type the following command exactly and press Enter:

DOS

```
ollama run gemma4:e2b
```

- c. **Wait for completion:** The terminal will show a download progress bar. It will pull down the multi-gigabyte model weights directly to your hard drive. Once finished, it will present a success message and an interactive prompt. Type `/exit` and press Enter to return to the normal command line interface.

SECTION 3: DOWNLOADING AND SETTING UP NODE.JS (The Backend Runtime)

The toolserver script requires a JavaScript environment to manage local web searches and scrapers.

1. **Go Online:** Navigate to the official Node website: <https://nodejs.org/>.
2. **Select Version:** Click the button labeled **LTS (Long Term Support)**. This version ensures absolute stability. Do not select the "Current" version.
3. **Install:** Double-click the downloaded installer file (e.g., `node-vxx.xx.x-x64.msi`).
4. **The Wizard Setup:** Click **Next**, accept the license agreement terms, and accept the default installation paths. Crucial: When it asks if you want to install tools for native modules, check the box, then click **Finish**.
5. **Environment Verification:**
 - a. Open a completely fresh Command Prompt window (`cmd`).
 - b. Type the following commands to ensure your operating system recognizes the installation paths:

DOS

```
node -v  
npm -v
```

- c. If both commands return version numbers, Node.js is successfully integrated.

SECTION 4: ASSEMBLING THE TOOLSERVER FILES

Now, you will build the workspace directory on your desktop to get the background scrapers running.

1. **Create the Folder:** Go to your Windows Desktop, right-click an empty space, select **New -> Folder**, and name it lowercase: `toolserver`.
2. **Establish the Configuration File (`package.json`):**
 - a. Open Windows Notepad.
 - b. Paste the following configuration block exactly as it is shown here:

JSON

```
{
  "name": "toolserver",
  "version": "1.0.0",
  "description": "Local Web Scraper Bridge for CephalopodAI",
  "main": "toolserver.js",
  "type": "module",
  "scripts": {
    "start": "node toolserver.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.19.2",
    "node-fetch": "^3.3.2"
  }
}
```

- c. *Why this matters:* The `"type": "module"` flag explicitly tells Node to parse your scripts using modern ES Module syntax. This completely eliminates typeless package warning loops and prevents performance overhead bottlenecks.
 - d. **Save File:** Click **File -> Save As**. Navigate to your Desktop/`toolserver/` folder. Change the "Save as type" dropdown menu to **All Files (.)**. Name the file precisely `package.json` and click save.
3. **Place your `toolserver.js` File:** Ensure your main `toolserver.js` script file is placed directly inside that same folder
(`C:\Users\barry\Desktop\toolserver\toolserver.js`).

SECTION 5: INSTALLING BACKEND DEPENDENCIES

With your files in place, you must download the modules that handle the web traffic and cross-origin security layers.

1. **Open the Directory:** Open your Command Prompt window (cmd).
2. **Navigate:** Type the following change directory command to target your desktop folder path exactly and hit Enter:

DOS

```
cd c:\Users\barry\Desktop\toolserver
```

3. **Run the Installer:** Type the following command and hit Enter:

DOS

```
npm install
```

4. **Verification:** Node will automatically read your `package.json` manifest file and download `express`, `cors`, and `node-fetch` directly into a newly created `node_modules` subdirectory.

SECTION 6: THE LAUNCH AND EXECUTION SEQUENCE

To use the system, you must run the background utility server followed by the frontend user workspace interface.

Step 1: Fire Up the Backend Toolserver

1. In your Command Prompt window (which should still be located at `c:\Users\barry\Desktop\toolserver`), type the execution command and hit Enter:

DOS

```
node toolserver.js
```

2. **Look for Success:** The terminal window must respond with the text message:

Plaintext

Server Active on 3001

3. **Crucial Rule: Do not close this command window.** Minimize it. If you close this window, the web scrapers die, and the AI will lose its ability to ingest live external information packets.

Step 2: Open the Front-End Cephalopod AI Core

1. Locate your primary interface frontend file:
CEPHLOPODAI_INTERFACE_Intellectual_Freedom_V2.0.html (or your latest V12 iteration file).
2. Right-click on the file icon, select **Open With**, and choose your primary browser window framework (e.g., **Google Chrome** or **Microsoft Edge**).
3. **System Verification:**
 - a. Look at the black-box chat log terminal on the left. It should immediately display initialization confirmation lines tracking back to the core setup.
 - b. Look at the top status banner. The built-in time code tracker will display your computer's exact internal system clock and date in real-time, alongside the active Cumulative Session Tokens counter.
4. **Test Your First Entry:** Type a message into the main prompt textarea box and click **Transmit**. The local Gemma model weights will instantly read the message array and return a direct text output to your viewport screen.

SECTION 7: DETAILED COMPONENT USAGE WORKFLOW

Now that the system is fully running on your computer, here is how you control the three main operations of the interface layout:

A. The Basic AI Interaction Loop

- **The Text Prompt:** Draft your research questions or manual coding inputs inside the Prompt Input field on the left.
- **Transmit:** Clicking **Transmit** wraps your current text alongside your complete session history array and pushes it to your local Ollama loop backport (<http://localhost:11434>). The AI responses append cleanly to your view with automated layout scrolling.
- **Net Search Integration:** If you need live real-time information from outside, type your specific search query inside the input box and click **Net Search** instead of Transmit. Your background toolserver fetches the raw search engine HTML layout data, runs it through the regex cleanup sifter to strip junk characters, and injects clean text lines into the chat stream as a `system_data` packet, which the local Gemma model analyzes instantly.

B. Workspace Management & The Cross-Panel Bridge

- **Persistent Canvas:** The massive right-hand panel is the **Canvas** workspace. This space is entirely independent of the chat layout stream, meaning your core manuscripts or complex engineering documents will never scroll away or get buried under conversation logs.
- **Send to Canvas:** Below every incoming response in your chat log sits a dedicated button. Clicking it instantly clones that specific AI answer and drops it straight over into your persistent Canvas pad, appending an unchangeable, secure timestamp to log when that text layout was finalized.
- **Send Canvas to Prompt:** If you are working on a long-form document or complex block of code inside your Canvas and want the AI to refine or debug it, click **Send Canvas to Prompt**. This clones the entire canvas scratchpad contents and drops it right into your left-hand input area so you can instantly send it back to the model for rapid updates.

C. The Session State Cartridge System

- **Save Session:** Clicking **Save Session** takes your entire current operational environment—including your full chronological chat history log, total computed session tokens, and file stamps—and packages it cleanly into a portable `.json` data file saved directly to your computer.
- **Load Session:** Clicking **Load Session** pulls open a file browser window. You can upload any previously saved `.json` file to instantly restore your exact conversational timeline, completely resetting the model's active working memory cache and protecting you from text drift.

This guide provides exactly what is needed for the GitHub release. It walks users directly from an empty machine straight through downloading, installing, folder configuration, execution commands, and full operational navigation.

Let's double-check this file layout. Are you ready to save this out as your master guide file, or do you want to adjust any of the installation steps before we move on to formatting your code repository elements?